General TTT

Program Design Discussion

This program is designed using an object-oriented approach to separate responsibilities and improve readability, testing, and scalability. The program supports multiple players (3–8) and a variable-sized board. The Game class controls the overall flow, including player setup, turn rotation, win/draw detection, and statistics tracking across multiple games. The Board class manages the grid using a single array, handles move placement, validates positions, and detects winning configurations. The Player class stores each player's name, assigned piece, and cumulative statistics (wins, losses, draws). The main function is minimal and simply creates a Game object and calls run() to begin execution.

Input validation is handled carefully to ensure robustness. Player names must contain exactly two alphabetic words. The number of players must be between 3 and 8. Board dimensions must fall within allowed ranges. Move input must follow the format of a letter followed by a number (e.g., A1) and must correspond to a valid, empty position. The game continues until a win or draw occurs, after which statistics are updated and the user is prompted to continue.

Data Structure Design Discussion

Player Data Structure
Each Player object contains:
- string firstName, string lastName: stores identity for prompts and display
- char piece: unique character assigned to each player
- int wins, int losses, int draws: cumulative statistics across games

This structure ensures that player data persists across multiple rounds and remains independent of board logic.

Board Data Structure
The board is implemented using:
- char cells[MAX_ROWS * MAX_COLS]: a single array representing the grid
- int rows, int cols: track board dimensions

A single array is used instead of a 2D array, with index mapping (row * cols + col). This allows efficient O(1) access and aligns with array-based techniques taught in data structures courses.

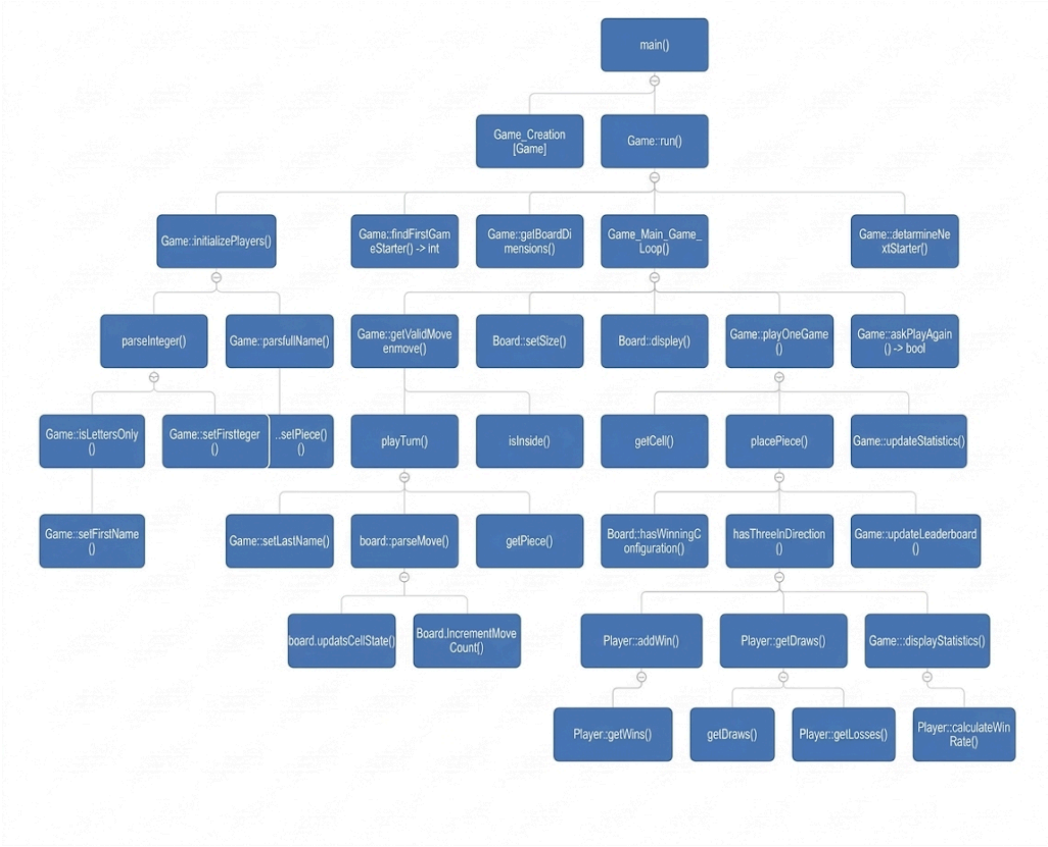
Game Control Structure
The Game class contains:
- Player players[MAX_PLAYERS]
- Board board
- int playerCount
- int totalGamesPlayed
- int starterIndex

This structure centralizes control logic, making it easy to manage turns, track game results, and rotate starting players.

Efficiency and Complexity Notes

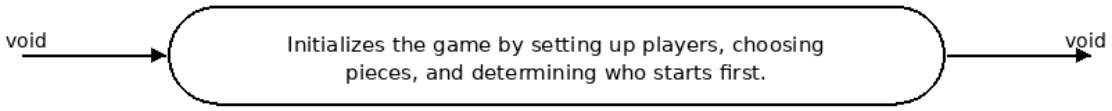
- Making a move is O(1) using direct array indexing
- Checking for a win is O(n·m) where n and m are board dimensions
- Checking if the board is full is O(n·m)
- Memory usage is constant relative to max board size and player count

Hiercharchy Chart:

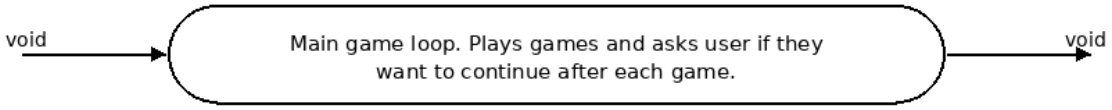


Procedure Specification

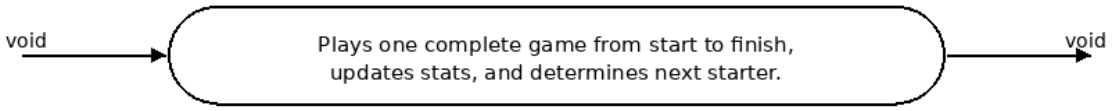
Function Game::setup



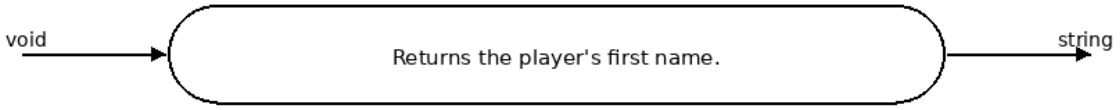
Function Game::run



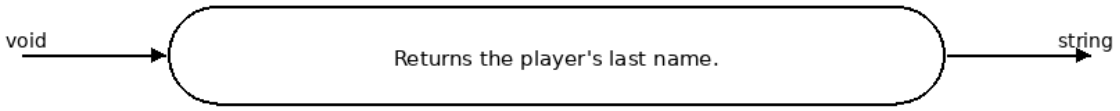
Function Game::playGame



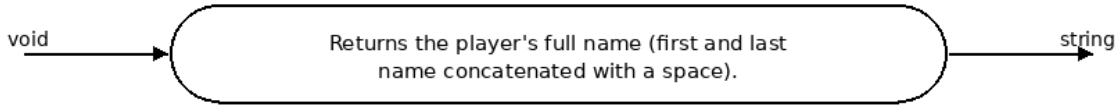
Function Player::getFirstName



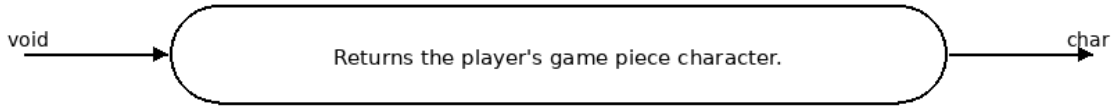
Function Player::getLastName



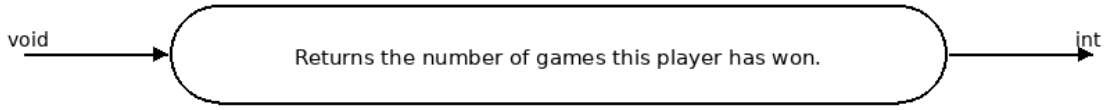
Function Player::getFullName



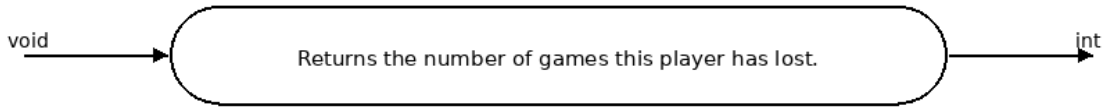
Function Player::getPiece



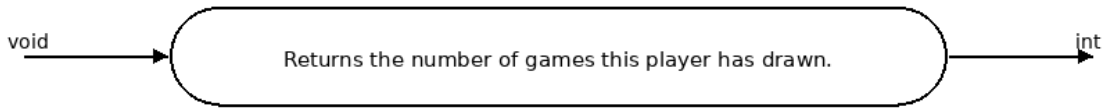
Function Player::getWins



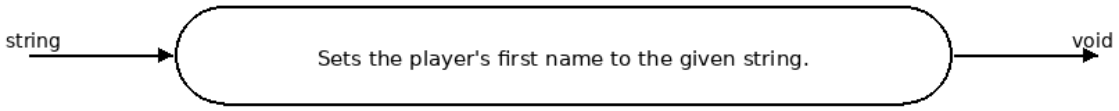
Function Player::getLosses



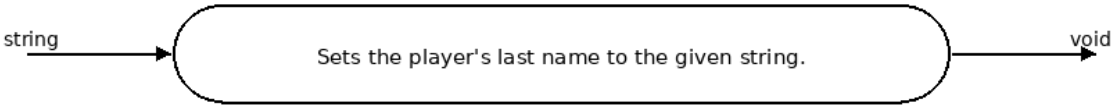
Function Player::getDraws



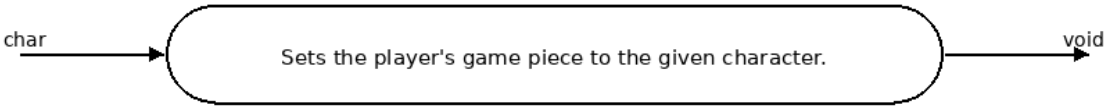
Function Player::setFirstName



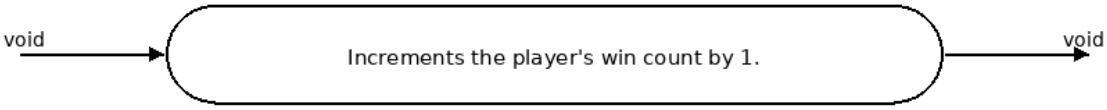
Function Player::setLastName



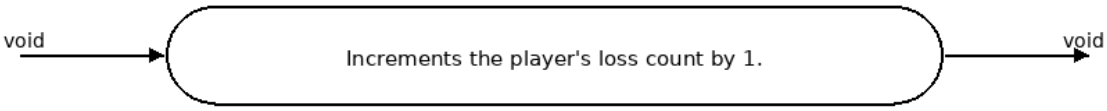
Function Player::setPiece



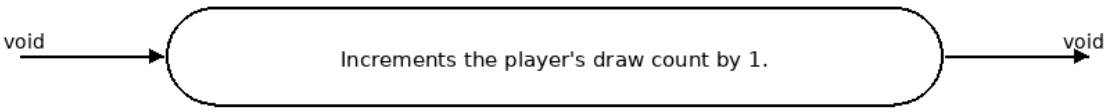
Function Player::addWin



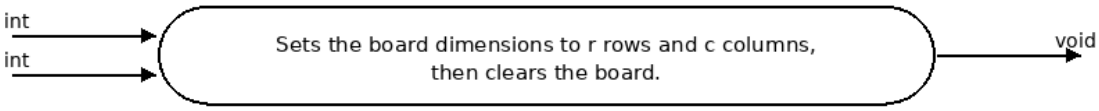
Function Player::addLoss



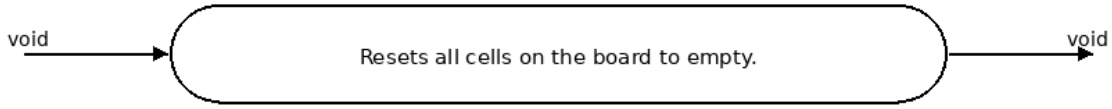
Function Player::addDraw



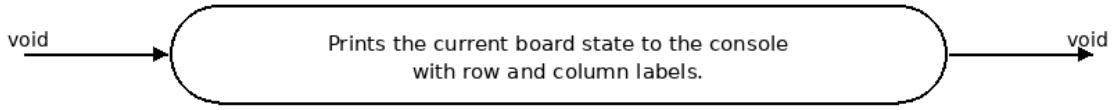
Function Board::setSize



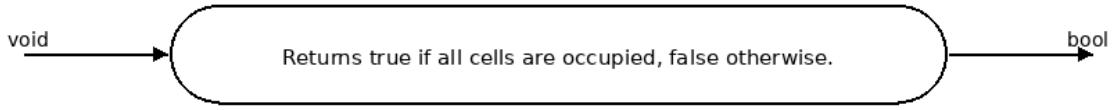
Function Board::clear



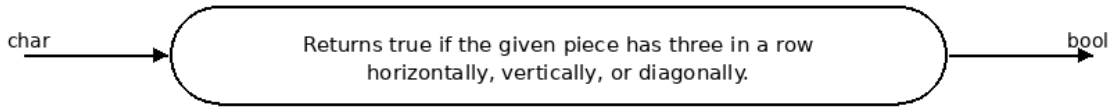
Function Board::display



Function Board::isFull



Function Board::hasWinningConfiguration



Test Case 1 — Valid Minimum Setup
Inputs: 3 players; Brent Foxworth, Ava Brown, John Doe; rows = 4; columns = 4
Justification: Confirms the program accepts the minimum valid number of players and minimum valid board dimensions.
Expected Output: Program accepts all players, displays a 4x4 board, and begins gameplay.

Test Case 2 — Valid Maximum Setup
Inputs: 8 players; eight valid names; rows = 11; columns = 14
Justification: Confirms the upper boundary for both player count and board size works correctly.
Expected Output: Program initializes all players, displays an 11x14 board, and starts the game.

Test Case 3 — Invalid Player Count Below Range
Inputs: 2 → 3
Justification: Ensures the program rejects player counts below the allowed minimum.
Expected Output: Program prints an error message and reprompts until a valid number is entered.

Test Case 4 — Invalid Player Count Above Range
Inputs: 9 → 5
Justification: Ensures the program rejects player counts above the allowed maximum.
Expected Output: Program prints an error message and reprompts until a valid number is entered.

Test Case 5 — Invalid Name (Contains Non-Letters)
Inputs: Br3nt Foxworth → Brent Foxworth
Justification: Verifies that names must contain only alphabetic characters.
Expected Output: Program rejects invalid name and reprompts for correct input.

Test Case 6 — Invalid Name (Missing Last Name)
Inputs: Brent → Brent Foxworth
Justification: Ensures both first and last names are required.
Expected Output: Program rejects input and reprompts until valid name is entered.

Test Case 7 — Invalid Rows Input
Inputs: rows = 3 → 4
Justification: Ensures row values must fall within the allowed range.
Expected Output: Program prints an error and reprompts until a valid value is entered.

Test Case 8 — Invalid Columns Input
Inputs: columns = 15 → 14
Justification: Ensures column values must fall within the allowed range.
Expected Output: Program prints an error and reprompts until a valid value is entered.

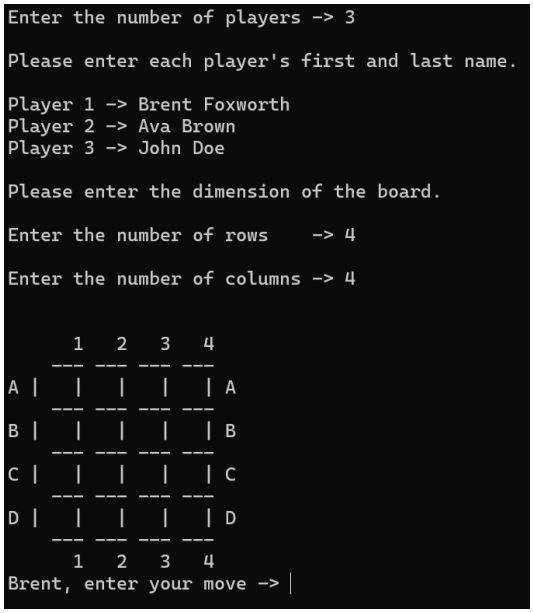
Test Case 9 — Invalid Move Format
Inputs: 5 → B2
Justification: Ensures moves must follow correct coordinate format.
Expected Output: Program rejects invalid format and reprompts until valid input is given.

Test Case 10 — Invalid Move (Out of Bounds)
Inputs: Z9 on a 4x4 board → C3
Justification: Ensures moves must be within board limits.
Expected Output: Program rejects move and reprompts until valid input is given.

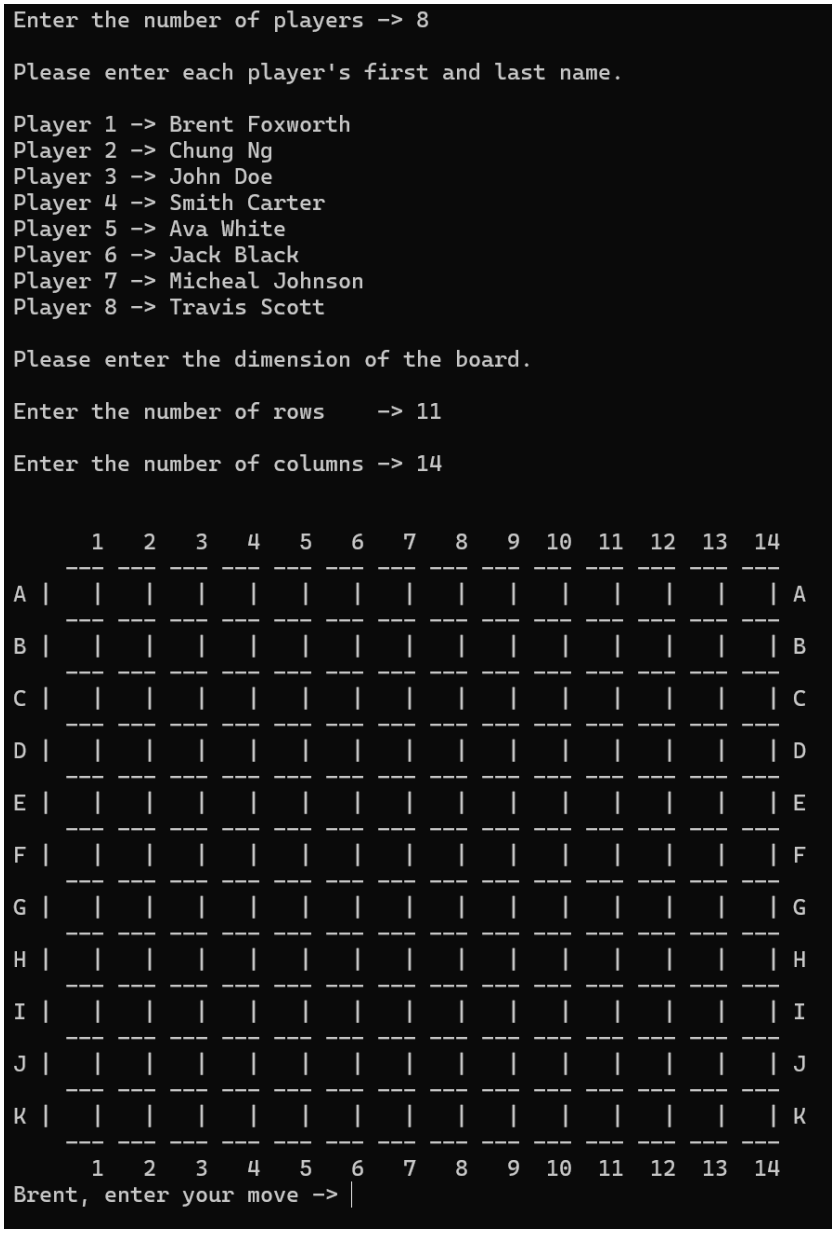
Test Case 11 — Win
Inputs: Same player occupies A1, A2, A3
Justification: Verifies win detection works correctly.
Expected Output: Program declares the player as winner and highlights the winning sequence.

Test Case 12 — Draw Condition
Inputs: Board fills with no three-in-a-row
Justification: Ensures the program correctly detects a draw.
Expected Output: Program prints draw message and updates statistics accordingly.

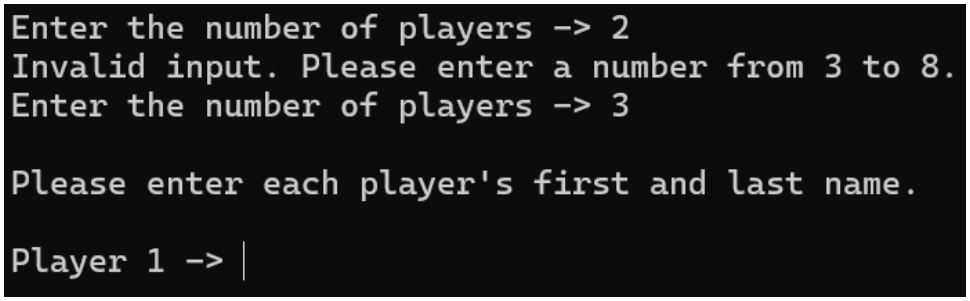
Test Case 1:



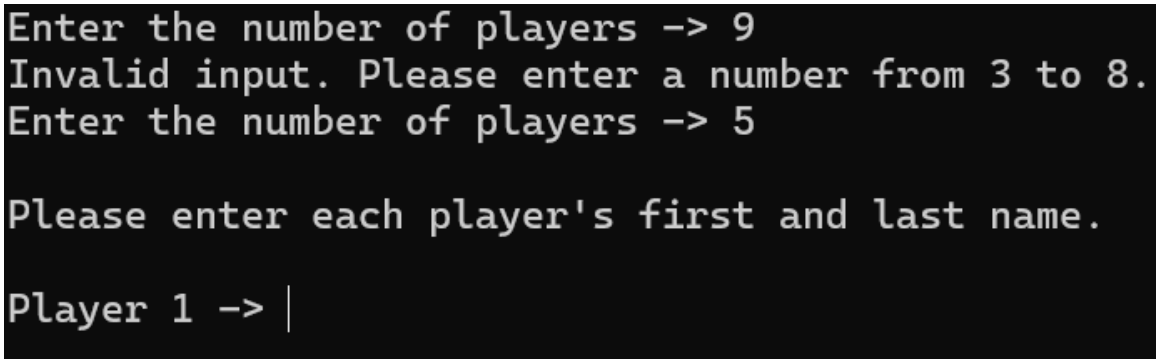
Test Case 2:


Enter the number of players -> 8
Please enter each player's first and last name.
Player 1 -> Brent Foxworth
Player 2 -> Chung Ng
Player 3 -> John Doe
Player 4 -> Smith Carter
Player 5 -> Ava White
Player 6 -> Jack Black
Player 7 -> Micheal Johnson
Player 8 -> Travis Scott
Please enter the dimension of the board.
Enter the number of rows -> 11
Enter the number of columns -> 14
1 2 3 4 5 6 7 8 9 10 11 12 13 14
A B C D E F G H I J K
Brent, enter your move ->

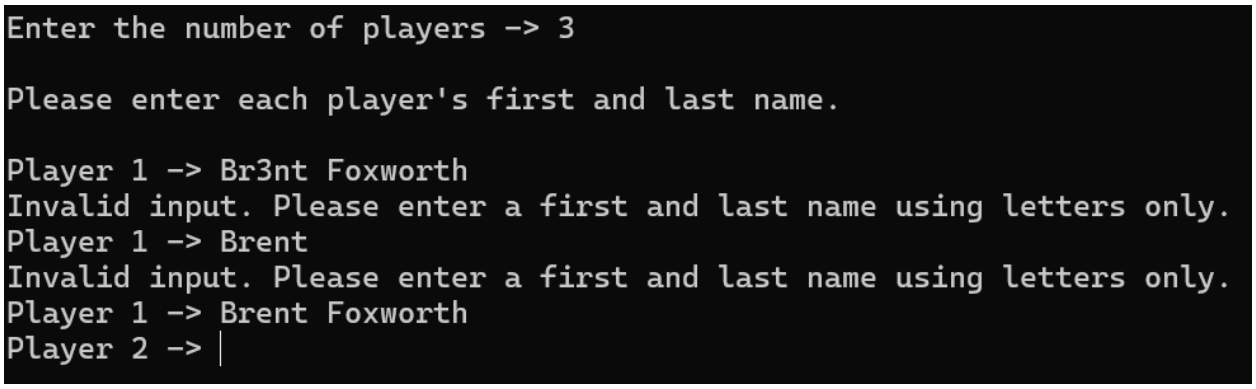
Test Case 3:



Test Case 4:



Test Case 5:



Test Case 6:

```
Player 1 -> Brent Foxworth
Player 2 -> Max Steel
Player 3 -> John Doe

Please enter the dimension of the board.

Enter the number of rows    -> 3
Invalid input. Please try again.

Enter the number of rows    -> 4

Enter the number of columns -> 15
Invalid input. Please try again.

Enter the number of columns -> 14


      1   2   3   4   5   6   7   8   9  10  11  12  13  14
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
A |   |   |   |   |   |   |   |   |   |   |   |   |   |   | A
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
B |   |   |   |   |   |   |   |   |   |   |   |   |   |   | B
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
C |   |   |   |   |   |   |   |   |   |   |   |   |   |   | C
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
D |   |   |   |   |   |   |   |   |   |   |   |   |   |   | D
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
      1   2   3   4   5   6   7   8   9  10  11  12  13  14
Brent, enter your move ->
```

Test case 8:

```
Brent, enter your move -> A15
Invalid input. Please try again.
Brent, enter your move -> E12
Invalid input. Please try again.
Brent, enter your move -> hello
Invalid input. Please try again.
Brent, enter your move -> 13
Invalid input. Please try again.
Brent, enter your move -> 1a
Invalid input. Please try again.
Brent, enter your move -> A1

      1   2   3   4   5   6   7   8   9  10  11  12  13  14
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
A | z |   |   |   |   |   |   |   |   |   |   |   |   |   | A
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
B |   |   |   |   |   |   |   |   |   |   |   |   |   |   | B
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
C |   |   |   |   |   |   |   |   |   |   |   |   |   |   | C
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
D |   |   |   |   |   |   |   |   |   |   |   |   |   |   | D
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
      1   2   3   4   5   6   7   8   9  10  11  12  13  14
Max, enter your move -> 
```

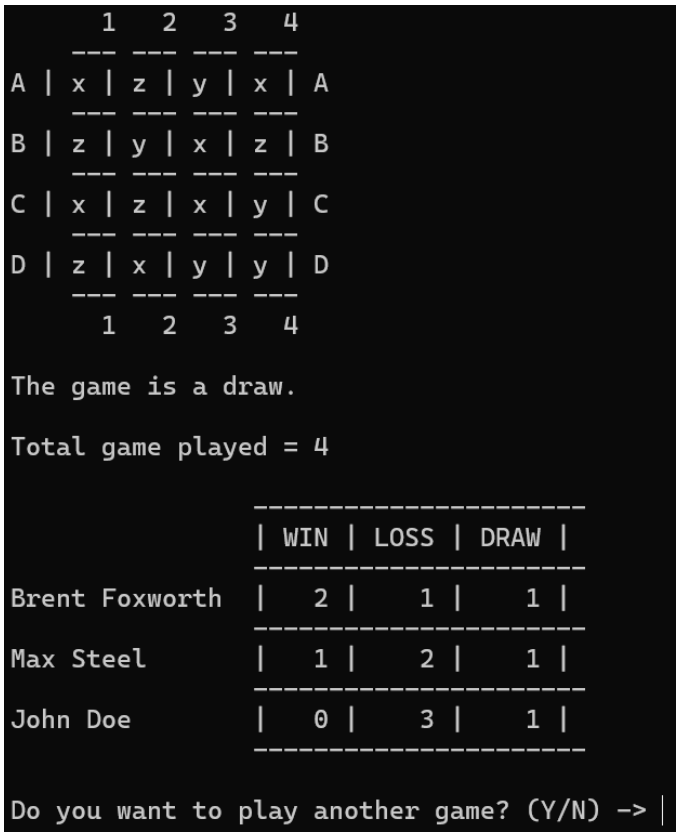
Test case 9:

```
Max, enter your move -> A1
Invalid input. Please try again.
Max, enter your move -> A2

      1   2   3   4   5   6   7   8   9  10  11  12  13  14
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
A | z | y |   |   |   |   |   |   |   |   |   |   |   |   | A
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
B |   |   |   |   |   |   |   |   |   |   |   |   |   |   | B
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
C |   |   |   |   |   |   |   |   |   |   |   |   |   |   | C
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
D |   |   |   |   |   |   |   |   |   |   |   |   |   |   | D
    --- --- --- --- --- --- --- --- --- --- --- --- --- ---
      1   2   3   4   5   6   7   8   9  10  11  12  13  14
John, enter your move -> 
```

Test case 10:

Test case 11:



Test case 12:

```
Total game played = 1

           ----------------------
           | WIN | LOSS | DRAW |
           ----------------------
Brent Fox  |   1 |    0 |    0 |
           ----------------------
John Doe   |   0 |    1 |    0 |
           ----------------------
Nick Dox   |   0 |    1 |    0 |
           ----------------------

Do you want to play another game? (Y/N) -> Hello
Invalid input. Please enter Y or N.

Do you want to play another game? (Y/N) -> 123
Invalid input. Please enter Y or N.

Do you want to play another game? (Y/N) -> Yes
Invalid input. Please enter Y or N.

Do you want to play another game? (Y/N) -> y
```